

ProtEC: A Transformer Based Deep Learning System for Accurate Annotation of Enzyme Commission Numbers

Azwad Tamir¹, Milad Salem², and Jiann-Shiun Yuan³

Abstract—The advancements in next-generation sequencing technologies have given rise to large-scale, open-source protein databases consisting of hundreds of millions of sequences. However, to make these sequences useful in biomedical applications, they need to be painstakingly annotated by curating them from literature. To counteract this problem, many automated annotation algorithms have been developed over the years including deep learning models, especially in recent times. In this work, we propose a transformer-based deep-learning model that can predict the Enzyme Commission numbers of an enzyme from full-scale sequences with state-of-the-art accuracy compared to other recent machine learning annotation algorithms. The system does especially well on clustered split dataset which consists of training and testing samples derived from different distributions that are structurally dissimilar from each other. This proves that the model is able to understand deep patterns within the sequences and can accurately identify the motifs responsible for the different enzyme commission numbers. Moreover, the algorithm is able to retain similar accuracy even when the training size is significantly reduced, and also, the model accuracy is independent of the sequence length making it suitable for a wide range of applications consisting of varying sequence structures.

Index Terms—Deep learning, enzyme commission numbers, machine learning, ProtBert, protein annotation, protein sequences, transformers.

I. INTRODUCTION

THE rapid development of sequencing technologies has made it practical and cost-effective to have large databases of protein sequences both in the private domain and also under publicly available open-source licenses. This large increase in the availability of protein sequences gave rise to protein databases like Uniprot [1] consisting of hundreds of millions of data points. And with every passing day, more than a hundred thousand proteins are being added to these continually growing global public protein databases [1]. However, in order to make

these protein databases useful to their full extent, they need to be correctly annotated by the scientific community. To help out with this problem, significant effort is put in by domain experts and curators to manually annotate these protein sequences, but this process is very slow and only about 0.03% of the proteins available in the public databases could be successfully annotated in this manner [1].

The pressure to find faster methods to annotate large volumes of proteins has resulted in the rise of numerous automated and semi-automated systems. Several types of annotation systems evolved over the years that used different techniques to label the proteins. One method was to compare the sequence of the target protein to other already annotated proteins to find similarities between them. The target was given the same annotation as the known protein if significant patches of sequence between them matched up with each other. BLAST [2] was one of the most successful tools that used this type of method to annotate protein sequences.

Another popular method for protein annotation that came later on used functional motifs to annotate target proteins. It assigned a particular function to a protein if it contains the motif associated with that function. Later versions of these models started using profile Hidden Markov models (HMMs) which made them more robust and able to find soft matches [3], [4]. These models computed the probability of a new sequence having a particular function if the probability went above a certain threshold. They are fast and efficient leading to many prominent protein databases like Pfam and InterPro using them to annotate their large protein databases [5], [6]. However, these models, despite being quick and accurate, do have their limitations. First, they compute the probability of each function motif individually thus being less efficient and accurate when there exist functional groups that share a common feature. Moreover, the signatures used to detect a particular function in a protein sequence often require involvement from human curators and are not fully automated, thus making the whole process somewhat slow and arduous. These limitations warrant the need for further study into more effective annotation systems in order to process a large number of proteins that are added to the databases every day.

Machine learning algorithms or more specifically a variant of it known as deep learning, which could be characterized as models having more than one hidden layer, have gained a lot of attention in recent times for their success in a wide range of different applications. Early deep learning models worked on

Manuscript received 12 December 2022; revised 28 August 2023; accepted 29 August 2023. Date of publication 4 September 2023; date of current version 26 December 2023. (Corresponding author: Azwad Tamir.)

Azwad Tamir is with the Department of Electrical and Computer Engineering, University of Central Florida, Orlando, FL 32826 USA (e-mail: azwad.tamir@ucf.edu).

Milad Salem and Jiann-Shiun Yuan are with the University of Central Florida, Orlando, FL 32816 USA (e-mail: milad73s@gmail.com; jian-shiun.yuan@ucf.edu).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TCBB.2023.3311427>, provided by the authors.

Digital Object Identifier 10.1109/TCBB.2023.3311427

applications in computer vision and machine translation but they soon expanded into other fields including medical imaging and bioinformatics.

The operation of a deep learning model generally consists of two stages. The first is the training step where labeled and semi-labeled data is used to learn the weights of the model followed by the inference step where the learned weights in the model are used to determine the label of an unknown sample. As the lower layers in a model are involved with more generalized functions, they could be shared between similar tasks. This method is known as transfer learning, where, instead of training from scratch for a new task, the weights of the model could be initialized from a similar pretrained application. After initialization, the model weights are trained on the new data in a step known as fine-tuning. This process produces better accuracy with less amount of training data, requiring lower training time and computational costs [7].

Following the success of deep learning models in various fields and applications, a number of studies have been carried out to apply them in protein functional prediction and classification tasks [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19]. Moreover, deep learning models have also been implemented on protein structure prediction [20], [21], [22], [23], [24] and protein design [25], [26], [27], [28], [29]. A milestone in protein annotation began when researchers started to feed large amounts of raw unstructured data into large deep-learning models. This spared the models from human biases, helped identify patterns that haven't been found yet, and alleviated the system from bottlenecks caused by the slow curation process. Recent advances in this area include DeepEC [30] which is an ensemble CNN model that annotates protein sequences with EC numbers. However, it could only annotate sequences that are less than a thousand units long and requires to have at least 10 datapoints of a single class to work properly. Other major contributions consist of CNN-based ensemble models to annotate unaligned protein domain sequences [31], using a transformer-based model to make an improvement in both accuracy and computational requirements [32], training different models to understand the structure of protein sequences [33] and building a transformer-based model to predict GO terms [34]. However, these studies do not deal with full-scale protein sequences which are often important for Bioinformatics applications. This problem has been counteracted in Proteinfer [35], which uses both a single and an ensembled CNN architecture to annotate full-scale protein sequences with GO terms and Enzyme Commission (EC) numbers.

The purpose of this study is to build upon Proteinfer [35] by proposing a transformer-based deep learning system that is able to take full-scale protein sequences and annotate them with EC numbers with better accuracy compared to Proteinfer, which we used as our primary benchmark. The proposed algorithm is able to predict all four stages of the hierarchical EC number of an enzyme given the input sequence.

The main novelties of our study are given below:

- Achieving state-of-the-art accuracy on EC number annotation of full-scale protein sequences.
- Using a single model instead of an ensemble of models to make the system lighter and easier to use.

- The model requires much less time and computational resources for training compared to other models.
- The proposed model is equipped with fully Automated hyperparameter tuning, which makes it easier to train, requiring no development or validation dataset.
- Better understanding of deep patterns within the protein sequences which is evident from the narrow gap in accuracy between the random and clustered dataset splits.
- The ability of the model to be able to perform very well even with a small amount of data.

The paper is organized into four sections. The first section provides an introduction to the project with relevant literature reviews. The second section describes the model architecture in detail and provides a description of the methodology. The third section shows the results obtained and compares them to other models. The fourth section discusses the various issues and technical information relating to the study. Finally, the fifth section concludes the work and provides further discussions with potential future research directions.

II. MODEL

This section describes the detailed model architecture and operating procedure along with the description of the dataset and some useful terminologies.

A. Enzyme Commission (EC) Numbers

Enzymes are a type of protein that catalyzes a biological chemical reaction. They are classified by the type of reactions they catalyze in the form of a hierarchical system known as the EC number which is made up of four integers separated by dots [36].

- The first number, which ranges from 1 to 7, signifies the broad type of reactions that the enzyme catalyzes.
- The second EC number represents the sub-class of the first EC number.
- The third number denotes the sub-subclass.
- Finally, the fourth number stands for the sub-subclass serial number for the enzyme.

So, the EC number of an enzyme serves as its primary identifier and is therefore of much importance to the scientific community.

B. Dataset

There are several open-source public repositories that host a large amount of protein data. The biggest of them is Uniprot [37] which contains a protein sequence database known as UniprotKB made up of two sections: SwissProt, which consists of more than five hundred thousand reviewed protein sequences, and TrEMBL, which is made up of around 200 million unreviewed sequences. The Swissprot part of UniprotKB is used as the dataset for this work as they are manually curated from the literature and reviewed by domain experts as opposed to TrEMBL which is unreviewed and is computationally annotated with the help of various automated systems like ARBA [37] and UniRule [38].

TABLE I
AVAILABILITY OF EC NUMBERS IN THE RANDOM SPLIT

Split	Proteins	EC1	EC2	EC3	EC4
Train	438,522	208,823	206,865	200,978	178,302
Dev	55,453	26,724	26,487	25,784	23,010
Test	54,289	25,892	25,665	24,992	22,183

TABLE II
AVAILABILITY OF EC NUMBERS IN THE CLUSTERED SPLIT

Split	Proteins	EC1	EC2	EC3	EC4
Train	182,965	87,520	86,667	84,273	74,943
Dev	180,309	86,230	85,537	83,156	73,641
Test	18,3475	87,661	86,784	84,299	74,892

Each protein sequence in the UniProt database contains several cross-referenced annotations like GO terms and EC numbers and they are verified using a 6-stage process with multiple credible sources like Pfam [6] and TIGRFAM [39]. The information from all these sources is fed into a mother database called InterPro [5], which could track the exact source of each annotation present in the SwissProt database.

The dataset consists of around 550,000 protein sequences which were split into training, development, and testing sets using two techniques. In the first case, the datapoints were split randomly between the three sets. For the second method called clustered split, UniRef50 [40] was used to split the data in such a way that sequences that are similar in structure are contained within a given split. This forces the training set to be made up of sequences that are structurally different from the development and testing sets ensuring that the test results outputted by the algorithm are fair and not contaminated by unrelated correlations between the sequences. Hence, the algorithm has to understand the exact patterns in the sequences responsible for each EC number to make accurate predictions. This type of method to create a clustered split produces a challenging test case and was the same method used in Proteinfer [35], which we used as our primary benchmark.

Both the random and clustered dataset splits of SwissProt were made publicly available in a data repository by the authors of proteinfer [35]. We used the exact dataset by downloading them from their website so that we can directly compare the test results outputted by our algorithm with that of Proteinfer [35]. The downloaded datapoints each contained a sequence and their corresponding GO terms and EC number annotations. We discarded the GO terms and only took the protein sequences and their corresponding EC numbers as only these are required in this work. The repository contained 438,522 datapoints among which 208,823 sequences are enzymes. These are filtered from the rest and are used for training and testing our model. A few of the enzymes did not have all four EC numbers available and the composition of enzymes in the dataset with each EC number in the training, development, and testing sets for the random and clustered splits are given in Tables I and II respectively. In the case of the random split, 80% of the sequences are put in the

training set with around 10% in the development and testing sets each. On the other hand, for the clustered split, the data is equally divided between the three splits making each split about one-third of the total dataset.

A number of other datasets were also used to compare the performance of Prot_EC with other models. Among these include the New dataset and the Cofactor dataset which was created by the authors of DEEPre [14] and downloaded directly from their repository. Also, the Uniprot and Held out dataset made in ECPred [9] was used to test the performance of Prot_EC with various other deeplearning models. The details of how these datasets were made are available in the DEEPre and ECPred publications [9], [14].

C. Transformers

Transformers are a type of deep learning algorithm that was first introduced in the paper entitled “Attention is All you Need”[41], where it was used for natural Language Processing (NLP) tasks. At the heart of the model were Attention blocks which the algorithm used for most of its computations. The model consisted of two parts: An encoder and a decoder. The encoder was responsible for converting input text into a vector representation while the decoder block converted the vector representation into an output text. The transformer is a sequence model as it takes the input tokens one by one and outputs another sequence in the same manner.

Later on, the transformer model was extended to other applications like the Vision Transformer [42], which is used to classify images. The different transformer types came in three categories. The first is the encoder models like ALBERT [43], BERT [44], DistillBERT [45], and ELECTRA [46] containing only an encoder. The second is the decoder models like CTRL [47], GPT-2 [48] and Transformer-XL [49], having only a decoder and the third is an encoder-decoder model like BART [50], and T5 [51] which consists of both an encoder and a decoder like the original transformer model [41].

D. ProtBert

The building blocks of our model are made up of a modified version of ProtBert [33], which is an encoder-only model pretrained on the BFD100 dataset consisting of 2,122 million protein sequences. The pretrained version of ProtBert was downloaded from the HuggingFace website [52]. During pretraining, portions of the sequence given by the masking probability were hidden and the model was trained to predict the masked tokens. This makes the model understand and learn deep patterns in the data along with the relationship between the different amino acids in the protein sequence [33]. The ProtBert module consisted of 30 hidden layers each having 1024 nodes and an intermediate hidden layer size of 4096. There were 16 attention heads and the model had 420 Million parameters in total. It was pretrained in [33] with the help of 128 Nodes and 1024 TPUs for 800 thousand steps on sequences with a maximum sequence length of 512 followed by another 200 thousand steps on sequences consisting of a maximum length of 2,000. A masking probability of 15% was used along with a Lamb optimizer. A

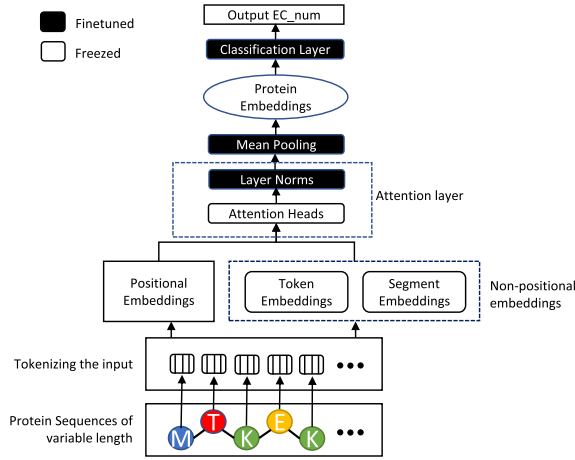


Fig. 1. Modified architecture of the ProtBert Module that was used in Prot_EC. The figure also indicates which layers were finetuned during the training process.

Algorithm 1: Data Preprocessing.

```

trainDf = pandas dataframe of training set;
for i ← 0 to len(trainDf) do
    if len(trainDf[i]['EC']) == 1 then
        EC_num.append(trainDf[i]['EC']);
        seq.append(seq[i]);
    if len(trainDf[i]['EC']) > 1 then
        EC_num.append(trainDf[i]['EC'][0]);
end
[EC1, EC2, EC3, EC4] = EC_num.split('.');
seq2 = str(EC1) + seq;
seq3 = str(EC1) + str(EC2) + seq;
seq4 = str(EC1) + str(EC2) + str(EC3) + seq;
[EC1_, EC2_, EC3_, EC4_] = labelEncoder([EC1, EC2,
    EC3, EC4])
[EC1x, EC2x, EC3x, EC4x] = OneHotEncoder([EC1_,
    EC2_, EC3_, EC4_]);
[seqx, seqx2, seqx3, seqx4] = tokenize([seq, seq2,
    seq3, seq4])
tokenizer.addSpecialTokens(str([EC1, EC2, EC3, EC4]))
Result: EC1x, EC2x, EC3x and EC4x are the output
        labels while seqx, seqx2, seqx3 and seqx4 are
        the input sequences

```

learning rate of 0.002 was selected with a weight decay rate of 0.01.

In order to modify the ProtBert module and use it in our work, we downloaded the pretrained weights and finetuned them on enzyme sequences. However, when it comes to large transformer models like ProtBert, even finetuning it requires a large amount of data and computational resources. To counteract this, we have implemented selective fine-tuning where some of the layers of the model are tuned while others are left to their pretrained values. This technique produces better accuracy on large models where the amount of data is limited [53], [54]. The basic architecture of the ProtBert model along with the selective finetuned layers is given in Fig. 1

E. Data Preprocessing

The data has to be preprocessed in a series of steps in order to make it suitable to be injected into the algorithm. The downloaded data [35] consisted of a series of protein sequences along with their corresponding EC numbers and GO terms. The sequences were filtered to separate out the proteins that were enzymes and had a corresponding EC number which was used as the target for the classifier models. The EC numbers for each sequence were separated based on the hierarchical stages and converted into one hot vector to make it suitable for the model. Next, the sequences were tokenized which involved assigning an integer number to each of the amino acids present in the data. Special tokens were added to the Tokenizer for the EC number labels so that they could be concatenated with the sequences and inputted into the model when necessary. The final tokenized dataset was made up of four arrays: The input_ids, which holds the tokenized sequences, the token_type_ids, which tells the algorithm the token type of each of the elements in the input_ids array, the attention mask, indicating the input segments to be masked during training and finally, the labels, which is an array with each element representing the one hot encoded version of the corresponding target label.

F. Training

After data preprocessing, comes the training step which involves teaching the algorithm the overall patterns within the enzyme sequences and the global and local relationships between the amino acids which determine which type of chemical reactions they catalyze as given in the EC numbers. A block diagram illustrating the training step is given in Fig. 2. The model consists of 4 ProtBert-like blocks (Prot_ECx), where each of which specializes in one stage of the EC numbers. Each of the blocks was modified by adding a classification header at the end where the number of nodes in the header was made to match the number of unique EC numbers present in each stage. Instead of training the blocks from scratch, the weights were initialized with pretrained values from the Huggingface repository where the model was trained on 2,122 million proteins from the BFD100 dataset which helps the model understand the general structure of proteins before they are trained on enzyme sequences.

$$Loss = -\frac{1}{N} \sum_i^N \sum_j^M y_{ij} \log(\hat{y}_{ij}) \quad (1)$$

Each ProtBert block was then trained (finetuned) on the enzyme sequences for 10 epochs with a gradient accumulated step of 32 on a single Nvidia RTX 2080Ti GPU taking around 40 hours for the training process to complete. This is much less compared to our main benchmark model (Proteinfer [35]) which requires 8 NVIDIA P100 GPUs running for 60 hours. No dropout was used as the model did not show signs of overfitting with mean pooler used as the pooler type. A multilabel cross-entropy loss function was used to calculate the training loss at each step which is given in (1). Here, N stands for the total number of training samples, M represents the number of classes

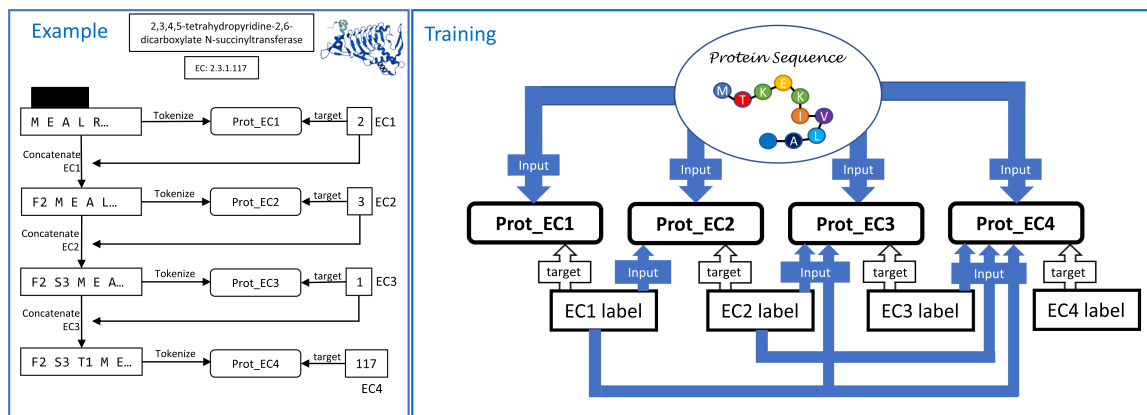


Fig. 2. Right: Data Flow block diagram during Training Stage of Prot_EC. Detailed architecture of each Prot_ECX module is given in Fig. 1. Each Prot_ECX block is responsible for calculating one stage of the EC number. The EC number of the previous stage is also concatenated with the sequence before feeding it to the next Prot_ECX block. Left: Example dataflow showing details about how the data is fed to the blocks during the training stage.

for a particular EC block, y is the true label, and $\hat{y}$ is the predicted label. During training, the positional and non-positional embeddings, and the attention layers were frozen, while the normalization layers, the pooler layers, and the classification headers were allowed to be finetuned. This ensured stability as the attention layers were already pretrained to hold the structural information of proteins. The initial learning rate was selected to be $5e-4$. However, this does not have any significant impact on the results as the learning rate was dynamically adjusted in each step of the training process.

The internal architecture of each Prot_ECX block is given in detail in Fig. 1. The input sequences are first fed to the tokenizer which tokenizes each amino acid into latent representation which is understood by the transformer algorithm. Next, the Positional and non-positional embeddings are generated and all the embeddings are summed together and inputted into the Attention layer which consists of multi-head attention blocks followed by the normalization layer. This outputs the amino acid embeddings which are then pooled together into the protein embeddings. Finally, the flow ends with a classification layer that predicts the Enzyme commission number.

Prot_EC, ProtBert, and typical BERT style models embed the input using three vectors; token embeddings, positional embeddings, and segment embeddings. Token embeddings are used to create a numeric representation of the sequence via taking each token (in this case each amino acid) and replacing it with a learnable vector. The order in which tokens appear in biological or natural language sequences is another important piece of information that should be included in the embedding. Positional embedding captures this order by using a learnable vector conditioned on the position of the token and adding it to the vector embedding. As a result of this addition, embedding vectors for tokens that are close together in the sequence are pushed in similar directions in the embedding space. The last embedding vector, i.e. segment embedding, is a remnant of the next sentence prediction function of BERT and not in use in ProtBERT-BFD. In this work, token and segment embeddings are referred to as non-positional embeddings. All the

TABLE III
ACCURACY AT DIFFERENT LAYER COMBINATIONS WHICH WAS USED TO DETERMINE WHICH EMBEDDING LAYERS TO FINETUNE DURING THE TRAINING PROCESS

EC_num	Layer finetune combinations				
	[0,1,0]	[1,1,0]	[1,0,0]	[0,0,0]	[0,1,1]
EC1	97.33	97.92	97.58	97.79	97.31
EC2	95.55	96.35	96.11	95.30	95.63
EC3	95.38	96.34	95.92	95.11	95.61
EC4	89.20	89.68	89.74	87.42	88.55

embeddings are then added together and pushed into the attention layers.

To determine which layers to freeze during the finetuning process, five different combinations of freezing positional, non-positional, and pooler layers were created and the model was trained using each combination to find out the one giving the best accuracy. The experimental results are provided in Table III. Here, the numbers in the square brackets illustrate which layers are frozen during the finetuning process. So, [1,1,0] represents the run where the positional and non-positional embeddings are frozen while the pooler layer is finetuned. On the other hand, the attention layers are always frozen as it would require very large computational resources and a very big dataset to train them. Whereas, the normalization layer and the classification layer are always finetuned as they are initialized from scratch and are task-specific.

Table III shows that finetuning the pooler layer while keeping the positional and non-positional embeddings frozen provides the best result. For each combination, the model was trained for 5 epochs on the random train dataset while the random development split was used to generate the Accuracy results. So the [1,1,0] combination was used for the final training process.

The first block named Prot_EC1 took the sequences as inputs with the first hierarchical stage of the EC numbers acting as the corresponding training target. On the other hand, the later blocks starting from Prot_EC2 took all the earlier stages of the EC numbers along with the sequence as inputs as illustrated

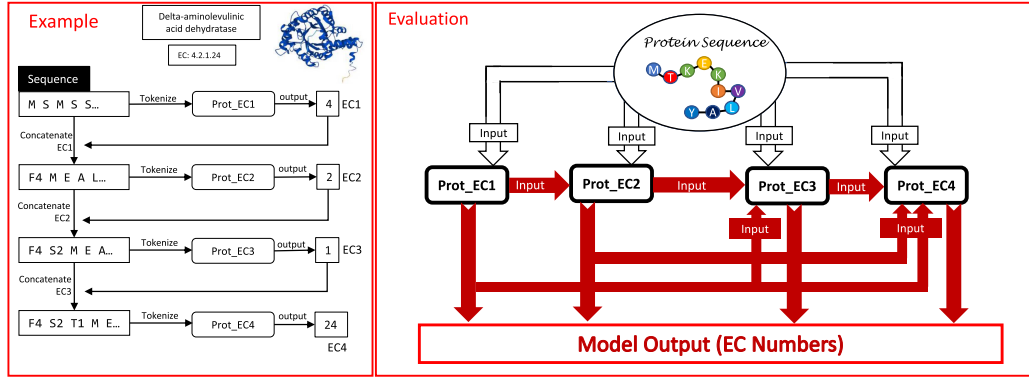


Fig. 3. Right: Data Flow block diagram during Evaluation Stage. The filled arrows mean predicted output from a previous Prot_ECX block while the non-filled ones stand for input sequences. The prediction of one block is concatenated to the next EC block sequence. Left: An example of the data flow during the evaluation stage.

TABLE IV
PERFORMANCE COMPARISON BETWEEN PROT_EC AND PROTEINFER MODELS FOR THE RANDOM AND CLUSTERED SPLIT DATASETS

Models	Parameters	Random split				Clustered split			
		EC1	EC2	EC3	EC4	EC1	EC2	EC3	EC4
Proteinfer	Accuracy(%)	97.69	97.32	97.10	95.61	89.93	88.02	86.73	82.23
	Precision	0.9857	0.9879	0.9902	0.9888	0.9680	0.9704	0.9770	0.9770
	Recall	0.9768	0.9715	0.9680	0.9504	0.8992	0.8778	0.8614	0.8094
	F1_score	0.9813	0.9796	0.9790	0.9692	0.9323	0.9218	0.9156	0.8853
	MCC	0.9644	0.9618	0.9617	0.9492	0.8772	0.8620	0.8569	0.8267
Proteinfer_EN	Accuracy(%)	97.71	97.43	97.23	95.75	89.42	87.53	86.51	82.48
	Precision	0.9918	0.9927	0.9934	0.9905	0.9829	0.9828	0.9845	0.9829
	Recall	0.9771	0.9733	0.9708	0.9544	0.8941	0.8738	0.8623	0.8205
	F1_score	0.9844	0.9829	0.9820	0.9721	0.9364	0.9251	0.9194	0.8944
	MCC	0.9705	0.9680	0.9671	0.9539	0.8869	0.8701	0.8647	0.8407
Prot_EC	Accuracy(%)	99.32	99.11	99.03	97.16	97.88	96.79	96.64	91.20
	Precision	0.9932	0.9910	0.9902	0.9718	0.9788	0.9678	0.9664	0.9133
	Recall	0.9931	0.9910	0.9901	0.9717	0.9787	0.9677	0.9663	0.9132
	F1_score	0.9931	0.9910	0.9901	0.9717	0.9787	0.9677	0.9663	0.9132
	MCC	0.9913	0.9894	0.9862	0.9709	0.9728	0.9618	0.9521	0.9107

in Fig. 2. This was done by concatenating the previous stage EC numbers with the protein sequence before tokenizing them into embeddings. Special tokens were used to tokenize the EC numbers of each stage. All previous EC stages had to be inputted into the next stage as the EC numbers are hierarchical and the previous stage values are also required to determine the label of the current stage. An example of the data flow during the training process is also given in Fig. 2 to illustrate the whole process.

G. Evaluation

The evaluation process of the model is provided in Fig. 3. It starts by feeding the protein sequences as inputs to the Prot_EC1 module. This outputs the predicted EC1 label which is then concatenated to the sequence input and inserted into the Prot_EC2 block after tokenization. Next, the second block, Prot_EC2 uses this information to predict the second EC number. In the case of the third and fourth Prot_EC blocks, all the previous predicted EC numbers are propagated to the model in order to make the prediction. For example, for Prot_EC4, the predicted output of

Prot_EC1, Prot_EC2, and Prot_EC3 are concatenated with the protein sequences and inputted into the model to get the fourth EC number. Finally, the predicted outputs of all four Prot_EC models are gathered and stitched together to represent the full EC number prediction corresponding to the given test sequence. An example of the data flow during the evaluation process is also given in Fig. 3 to illustrate the whole process.

III. RESULTS

This section presents the results and analysis of the presented algorithm Prot_EC, along with comparisons with other models and methods.

A. Performance Comparison

The overall test accuracies along with the Precision, Recall, F1 scores, and MCC scores of Prot_EC, Proteinfer, and Proteinfer_EN on the different EC numbers on both the random split and clustered split datasets are provided in Table IV. The Prot_EC is the model proposed in this study whereas Proteinfer

is a CNN-based protein annotation tool developed in [35] and Proteinfer_EN refers to an ensemble model consisting of multiple Proteinfer base models. On average, the accuracy scores of the models trained on the random split are greater than that of the clustered split. This is expected as it is more challenging with the training split and testing split samples derived from different clusters making them dissimilar in structure. Other trends in the data show that the accuracies fall for successive EC numbers with EC1 performing the best and EC4 showing the worst results. This happens as the number of classes increases for successive EC numbers with EC1 only having 7 classes whereas, EC4 has 402 classes. Also contributing to this trend is the fact that the EC numbers are hierarchical so any errors in the previous EC numbers are propagated to the later ones making the prediction of later values more difficult.

When comparing the different models, it is seen that Proteinfer has the lowest accuracy scores for all the EC numbers in RN split and only EC4 in CS split while the ensemble model Proteinfer_EN has the worst scores for the first three EC numbers in CS split. On the other hand, Prot_EC is seen to outperform the other benchmark models in all the EC numbers in both the dataset splits with the difference margins being much greater for the clustered split compared to the random. This shows that Prot_EC is better at understanding the detailed patterns within the enzyme sequences and the relationships between amino acids. Also, Transformers being primarily sequence-based algorithms gives them an edge over other types of deep learning models when it comes to working on very long sequences.

The F1_scores and Matthews Correlation Coefficient (MCC) scores follow a similar trend to the accuracy values with the random split results generally being higher for a model compared to that for the clustered split. The Proteinfer model shows the worst F1_scores in general, followed by the ensemble Proteinfer model, while Prot_EC gives a better score for most of the EC numbers on both the splits. The only exception is the fourth EC number for the random split, where the Proteinfer_EN has a slight advantage over Prot_EC in terms of the F1_score. However, this is not seen for the clustered split, where Prot_EC shows better F1_scores for all the EC numbers in both the dataset splits. Also, the trend of Prot_EC having better margins when it comes to the clustered split compared to the random split is seen for MCC and the F1_scores as well and confirms that Prot_EC shows better results compared to the benchmark models in terms of performance.

When we look at the precision and recall scores, it is seen that the benchmark models have a much higher precision score compared to the recall scores whereas, in the case of Prot_EC, the values are nearly equal for all the EC numbers across both the dataset splits. So, most of the predicted results of the benchmark models were true labels but it also did not pick up many of the actual positives in the datasets. The recall values could be buffed up by increasing the detection threshold of Proteinfer, but this again would decrease the precision value, so eventually, there would be minimal impact on the F1_scores. Moreover, the threshold value acts as an extra hyperparameter, and the need to tune it puts an extra burden on the end user decreasing the user-friendliness of the tool. Also, the threshold parameter

depends on the test dataset and its optimum value for each case is unknown to a user with an unlabeled test set. Prot_EC, on the other hand, does not require any extra threshold tuning to balance out the precision and recall values.

The test the performance comparison of Prot_EC and Proteinfer on non-homologous protein sequences, several different test sets were created with the help of CDHit [55], [56]. To prepare this, the training set was kept the same as in the random split dataset while the test set was curtailed to include only those sequences that are similar to the training set sequences by no more than a certain threshold. Six test sets were created with thresholds ranging from 90% to 40%. Here, a dataset made with a threshold of 40% similarity means that there are no sequences between the training and test sets that are more than 40% similar to each other. Table V shows the accuracy and F1 score of Prot_EC and Proteinfer on the various test sets created. The results show similar trends in both accuracy and F1 score where Prot_EC is able to hold out its performance much better compared to Proteinfer as the maximum allowed similarity is decreased from 90% to 40% with a large gap in performance between the two models occurring at the 40% threshold mark. This shows that Prot_EC is successful in predicting EC numbers even with very dissimilar training and test sets which indicates its ability to determine deep patterns within the protein sequences.

Next, the performance of Prot_EC was compared with some other popular deeplearning EC prediction models. Table VI shows the Accuracy, Cohen's Kappa score, Precision, Recall, and F1 scores on the New dataset created in DEEPre [14]. It has 22,168 protein sequences that were collected from UniProt SwissProt (released on 9/7/2016) database followed by a 40% similarity threshold sweep with CDHit [55] to only keep the low-homology enzyme sequences. A five-fold cross-validation method was used to evaluate the model performance. The results show Prot_EC is able to outperform the other models in most of the metrics. However, the margin of performance increase is lower compared to other datasets due to the smaller size of this dataset. Comparison for EC3 and EC4 classifications could not be made as only EC1 and EC2 results were available for these models.

Table VII shows the cross-dataset evaluation of Prot_EC compared to the other models on the primary EC number (EC1). Here, the models were trained on the DEEPre New dataset but were tested on the cofactor dataset which was an independent, non-homologous, and non-overlapping dataset collected from a separate source(PDB). Details of how the dataset was created could be found in DEEPre [14]. The results show Prot_EC outperforming the other models with a better margin as compared to Table VI. So, Prot_EC is better at predicting EC numbers where the train and test sets are more dissimilar from each other indicating that it's better at identifying patterns within the protein sequences.

Finally, Table VIII shows the comparison of Prot_EC with other models on the ECPred [9] Held out dataset. The training set was extracted from the UniProt SwissProt (release:2017_3) database while the test set consists of Enzymes that were annotated in later versions of Uniprot. This made the test dataset very challenging as it contained very little similarity with the training

TABLE V
ACCURACY AND F1 SCORE OF PROTEINFER AND PROT_EC MODELS WITH TRAIN AND TEST SPLIT OF DIFFERENT SIMILARITY INDEX

Models	Similarity	Accuracy				F1_score			
		EC1	EC2	EC3	EC4	EC1	EC2	EC3	EC4
ProteinFER_EN	0.9	95.63	95.09	94.68	91.77	0.9704	0.9674	0.9654	0.9459
	0.8	93.99	93.23	92.66	88.74	0.9593	0.9549	0.9521	0.9261
	0.7	91.68	90.67	89.87	84.64	0.9433	0.9374	0.9334	0.8985
	0.6	87.79	86.24	85.00	77.50	0.9160	0.9066	0.8999	0.8487
	0.5	80.79	78.31	76.23	64.60	0.8647	0.8495	0.8380	0.7490
	0.4	70.53	66.64	63.51	47.05	0.7823	0.7562	0.7360	0.6018
Prot_EC	0.9	98.77	98.44	98.20	94.83	0.9876	0.9843	0.9819	0.9481
	0.8	98.32	97.89	97.60	93.25	0.9830	0.9787	0.9759	0.9324
	0.7	97.77	97.16	96.75	91.17	0.9776	0.9715	0.9675	0.9116
	0.6	96.84	95.84	95.26	87.26	0.9682	0.9582	0.9524	0.8725
	0.5	95.10	93.42	92.63	80.64	0.9509	0.9342	0.9262	0.8062
	0.4	92.24	89.99	87.98	69.92	0.9223	0.9000	0.8799	0.6992

Here, CDHit was used to make the training and test sets of various similarities. The results show Prot_EC achieves better accuracy and F1_scores in all the different similarity values.

TABLE VI
PERFORMANCE METRICS OF DIFFERENT MODELS ON THE DEEPRE NEW DATASET WITH FIVE-FOLD CROSS-VALIDATION

Models	EC1					EC2				
	Accuracy	Kappa Score	Precision	Recall	F1_score	Accuracy	Kappa Score	Precision	Recall	F1_score
DEEPre	94.44	0.926	0.947	0.912	0.928	92.95	0.914	0.785	0.756	0.766
EzyPred	90.31	0.870	0.905	0.870	0.885	87.85	0.841	0.790	0.709	0.737
SVM-PsePSSM	83.34	0.769	0.913	0.753	0.812	84.99	0.795	0.831	0.677	0.732
NN-Row	88.07	0.840	0.830	0.807	0.817	89.30	0.861	0.661	0.615	0.629
Prot_EC	95.75	0.942	0.954	0.942	0.948	93.15	0.919	0.821	0.822	0.818

TABLE VII
EVALUATION OF PRIMARY EC NUMBER(EC1) PREDICTIONS OF DIFFERENT MODELS ON THE COFACTOR DATASET

Models	Accuracy	Kappa Score	Precision	Recall	F1_score
DEEPre	85.52	0.797	0.869	0.867	0.865
EFICAz	80.46	0.757	0.841	0.729	0.780
EzyPred	70.76	0.644	0.804	0.643	0.710
SVM-Prot	71.14	0.313	0.656	0.331	0.434
Prot_EC	89.39	0.860	0.907	0.905	0.906

TABLE VIII
COMPARISON OF PROT_EC WITH OTHER MODELS ON THE ECPRED HELD OUT DATASET

EC_num	Models	Precision	Recall	F1_score
EC1	ProtFun	0.15	0.10	0.12
	EzyPred	0.16	0.13	0.15
	EFICAz	0.69	0.30	0.42
	DEEPre	0.67	0.40	0.50
	ECPred-wne	0.34	0.48	0.40
	ECPred	0.54	0.43	0.48
	Prot_EC	0.89	0.87	0.88
EC2	EzyPred	0.13	0.10	0.11
	EFICAz	0.33	0.07	0.11
	DEEPre	0.07	0.25	0.11
	ECPred	0.35	0.20	0.26
	Prot_EC	0.45	0.55	0.49
EC3	DEEPre	0.14	0.03	0.05
	ECPred	0.31	0.17	0.22
	Prot_EC	0.39	0.43	0.41

set sequences. The results show Prot_EC performing significantly better than DEEPre and ECPred as well as the other models. This further proves that Prot_EC is better at predicting EC numbers when the training and test sets sequences are more dissimilar from one another. The substrate level EC number (EC4) comparison has not been made here as the test set contained too few EC4 labels to evaluate reliable performance metrics.

B. Confusion Matrix

Fig. 4 shows the confusion matrices of the first three EC_num classifiers on both the random and clustered splits. It shows the number of samples in each of the classes and how they were predicted by our model. The diagonal elements represent the number of test samples in each class that was correctly classified whereas, the off-diagonal elements were the wrongly predicted samples. The concentration on the diagonal elements is much more prominent compared to the rest for both the random and clustered splits confirming the high accuracy of the model. Also, the numbers on the diagonals for CT split are bigger than that of RN split even though the accuracy of RN is greater. This happens because the total size of the test set of CT is much larger than that of RN.

The matrices show similar trends for the misclassified samples in both splits. For example, in EC1_random, several samples belonging to class 3 were misclassified as class 2 and vice versa, and the same trend is also seen for the clustered split EC1 matrix. These similarities in trends are also present for the second and third EC number classifiers. This suggests that many of the classes are similar to each other and are difficult to differentiate during inference.

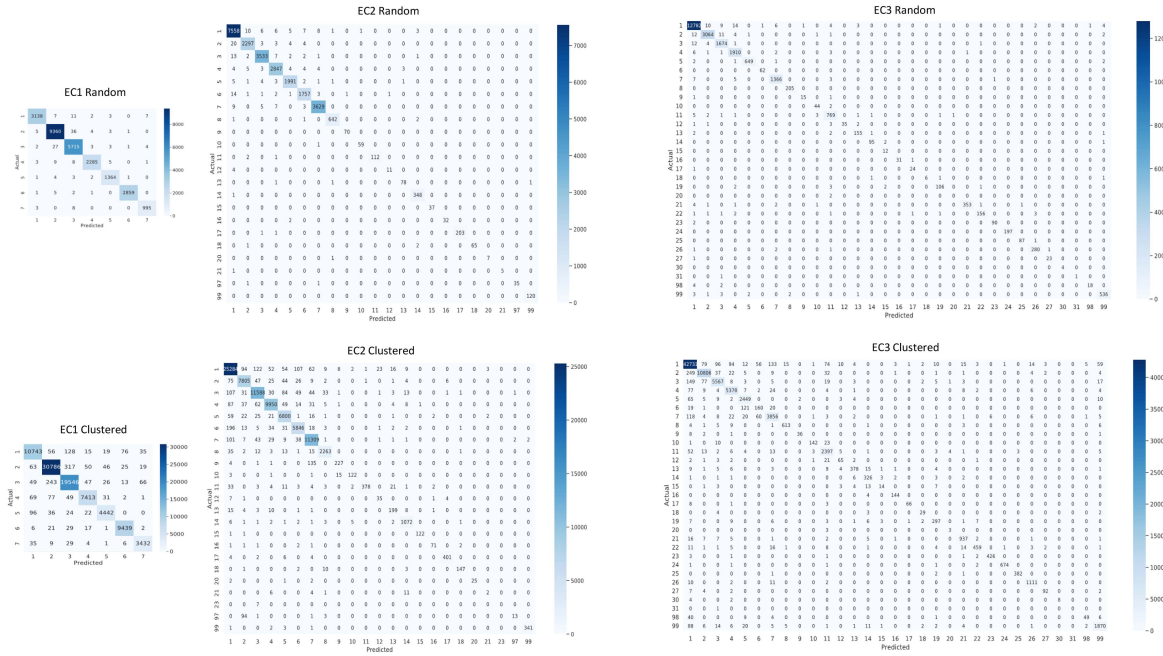


Fig. 4. Confusion Matrices for EC1, EC2 & EC3 classifiers on both dataset splits.

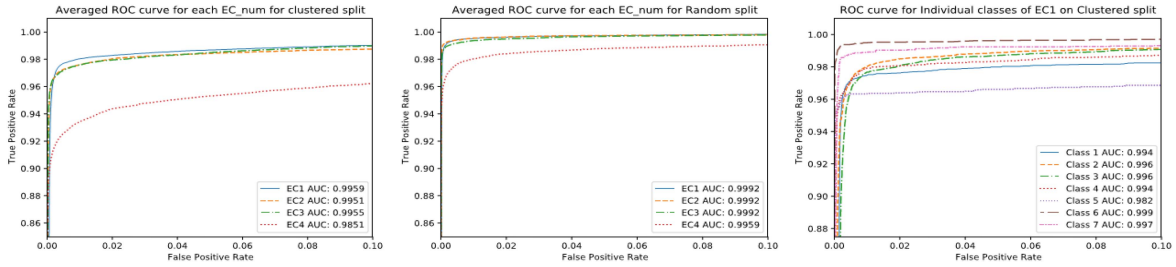


Fig. 5. ROC curves of various classifiers in Prot_EC for the random and clustered datasets.

C. ROC Curves and AUC

Next, the ROC curves for the Prot_EC model trained on the random and clustered splits are shown in Fig. 5. The curves show how well the model is able to separate the positive and negative labels at different threshold values. In an ideal case, the curve would take the form of an inverted L with the entire area falling under the curve and this denotes a 100% separability between the positive and negative samples. The third plot shows the individual class separability of the first EC number for the clustered split along with the area under the curve (AUC) for each class. This shows that it is easier to separate some classes compared to others. Another factor contributing to this is the size of the training sample for each class. It is seen that classes that have more samples in the training set tend to do better in the ROC curve compared to the ones with lower training data. For example, 36% of the training samples belong to class 2 and it shows a very prominent ROC curve whereas, only 5% belong to class 5 and is reflected by its worse shape and lower AUC score.

The first and second plots show the ROC curves and AUC scores for the different EC number classifiers in the case of

the clustered split and the random split datasets respectively. For the random split, the first three EC_num classifiers show very high AUC values along with sharp ROC curves while the classifier for the fourth EC number has a slightly lower score. A similar trend is also seen for the clustered split with the EC1 classifier showing very high AUC scores followed by the second and third EC number classifiers with EC4 having the lowest score. However, in the clustered split, EC4 is trailing behind the others with a larger margin compared to the random split. This could be explained by the fact that the clustered split dataset has a lower training size than that of the random split and this combined with a large number of classes in EC4 caused some of the classes to have a very low number of training data eventually adding to the difficulty in differentiating between these classes.

D. STS Analysis

The shrinking training set (STS) study analyzes the impact of reducing the size of the training set on the test accuracy of the model. Fig. 6 shows the test accuracies of the EC_num models with different training sizes on the random split. The

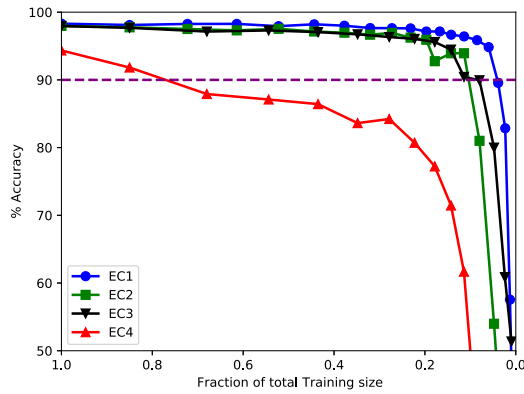


Fig. 6. Test Accuracies of Prot_EC at different Training set sizes derived from the random dataset.

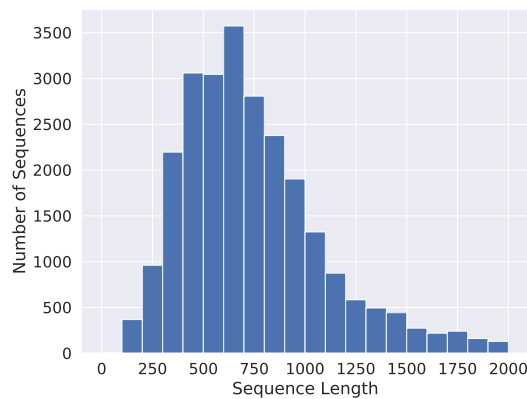


Fig. 7. Distribution of sequence Length in the Random split test set.

shrinking training sets are created by splitting the training data in a stratified manner into two parts, where one part is kept and the other one is discarded. The model is then trained on the modified training data for 3 epochs and the splitting step is repeated to get the next training set. The results show the algorithm is very resilient to shrinking training set size. The first three EC_num classifiers are able to hold comparable accuracy scores with only 10% of the total training size while the fourth one maintains a value above 85% with the training set reduced to half its original number. The lower performance of EC4 compared to the other three EC numbers could again be contributed to its large number of classes. This causes some of the classes to end up with a very limited number of samples diminishing the model's ability to correctly classify them.

E. Sequence Length Statistics

Fig. 7 shows the distribution of sequence lengths of enzymes for the random split test set up to 2000. Similar distribution could also be seen for the training and development sets and also for the whole dataset in general. There were a few samples that go beyond 2000, but their frequency keeps getting smaller as the sequence length increases so they are not included in the histogram. The frequency of samples seems to peak at around 750 units and the value diminished at a near-exponential rate making the bulk of the samples fall below a sequence length of 1500.

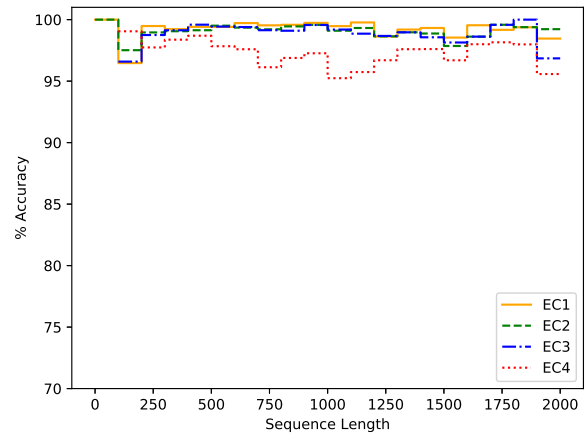


Fig. 8. Average test accuracy of Prot_EC for the random split at different sequence lengths.

Next, the average test accuracy for the random split is plotted against sequence length in Fig. 8 to show that the model's performance is not dependent on the size of the enzymes. The average accuracy is calculated by summing over all the correctly predicted samples in a range spanning every 100 units and dividing it by the total frequency of samples in that range. The graph shows steady accuracies up to a length of 1000, after which it showed slightly degraded performance. This occurs as the model was trained with sequences that were truncated to 1000 elements due to limitations on hardware availability in hand. However, the model does not have any limitations in terms of sequence size and could accommodate the training of sequences of any length. As a result, users who have access to more powerful GPUs could retrain the model on non-truncated sequences to get ever better performance.

IV. DISCUSSION

This section contains discussions relating to various issues and technical information regarding the study. During the training phase of the model, the training loss and evaluation loss were monitored in every epoch as the model was trained. Both losses followed similar trends and became saturated at around 10 epochs. The successive eval accuracies remained constant and did not decrease indicating that the model had enough countermeasures to stop overfitting so, the number of training epochs in this model is not a sensitive hyperparameter that needs to be carefully tuned making the whole process more robust.

There are a small number of Enzymes in the dataset that were associated with more than one set of EC numbers. However, as this number was very small (2.54%), the first set of EC numbers was extracted and used as the corresponding label to train and evaluate our model. The Proteinfer benchmark model on the other hand outputted multiple sets of EC numbers on some occasions. In these cases, the model is taken to have made the correct prediction even if one of its outputted EC sets matched any of the sets in the true labels. This makes sure that the accuracies of the benchmark models are not underestimated in any case.

The model was trained on sequences that were truncated to a length of 1000 due to the limitations in the available computational hardware. However, the model is able to operate with sequences of any length without any modifications to its architecture and could therefore be trained to obtain even better accuracy and performance if better computational hardware is available.

A. Conclusion

A transformer-based deep learning model named Prot_EC is proposed in this work which shows better performance in terms of accuracy and F1_scores compared to other full sequence deep learning models [35], especially for the clustered split. The model provides automatic hyperparameter tuning with balanced precision and recall values making it easy to use. Prot_EC is also able to retain its performance even with a fraction of its total training size and its accuracy is independent of sequence length making it suitable for enzymes of any length.

The results show that Prot_EC is able to perform very well with low homology train and test splits compared to other models. This part could be repurposed and used to predict other annotations in the protein universe with minimal modifications to the overall model structure. This makes it suitable to be expanded to predict multiple annotations using a single large model adding to the overall usage of the algorithm and is a potential and exciting area of future work.

REFERENCES

- [1] T. U. Consortium, "UniProt: A worldwide hub of protein knowledge," *Nucleic Acids Res.*, vol. 47, no. D1, pp. D506–D515, Nov. 2018. [Online]. Available: <https://doi.org/10.1093/nar/gky1049>
- [2] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, "Basic local alignment search tool," *J. Mol. Biol.*, vol. 215, no. 3, pp. 403–410, Oct. 1990.
- [3] A. Krogh, M. Brown, I. S. Mian, K. Sjölander, and D. Haussler, "Hidden markov models in computational biology: Applications to protein modeling," *J. Mol. Biol.*, vol. 235, no. 5, pp. 1501–1531, Feb. 1994.
- [4] S. R. Eddy, "Profile hidden Markov models," *Bioinformatics*, vol. 14, no. 9, pp. 755–763, Oct. 1998. [Online]. Available: <https://doi.org/10.1093/bioinformatics/14.9.755>
- [5] M. Blum et al., "The InterPro protein families and domains database: 20 years on," *Nucleic Acids Res.*, vol. 49, no. D1, pp. D344–D354, Jan. 2021.
- [6] S. El-Gebali et al., "The Pfam protein families database in 2019," *Nucleic Acids Res.*, vol. 47, no. D1, pp. D427–D432, Oct. 2018. [Online]. Available: <https://doi.org/10.1093/nar/gky995>
- [7] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *J. Big Data*, vol. 3, no. 1, May 2016, Art. no. 9. [Online]. Available: <https://doi.org/10.1186/s40537-016-0043-6>
- [8] M. Kulmanov, M. A. Khan, and R. Hoehndorf, "DeepGO: Predicting protein functions from sequence and interactions using a deep ontology-aware classifier," *Bioinformatics*, vol. 34, no. 4, pp. 660–668, Oct. 2017. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btx624>
- [9] A. Dalkiran, A. S. Rifaioğlu, M. J. Martin, R. Cetin-Atalay, V. Atalay, and T. Doğan, "ECPred: A tool for the prediction of the enzymatic functions of protein sequences based on the ec nomenclature," *BMC Bioinf.*, vol. 19, no. 1, Sep. 2018, Art. no. 334. [Online]. Available: <https://doi.org/10.1186/s12859-018-2368-y>
- [10] R. Cao, C. Freitas, L. Chan, M. Sun, H. Jiang, and Z. Chen, "ProLanGO: Protein function prediction using neural machine translation based on a recurrent neural network," *Mol.*, vol. 22, no. 10, Oct. 2017, Art. no. 1732.
- [11] J. J. Almagro Armenteros, C. K. Sønderby, S. K. Sønderby, H. Nielsen, and O. Winther, "DeepLoc: Prediction of protein subcellular localization using deep learning," *Bioinformatics*, vol. 33, no. 21, pp. 3387–3395, Jul. 2017. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btx431>
- [12] A. S. Schwartz et al., "Deep semantic protein representation for annotation, discovery, and engineering," *bioRxiv*, 2018. [Online]. Available: <https://www.biorxiv.org/content/early/2018/07/10/365965>
- [13] A. Sureyya Rifaioğlu, T. Doğan, M. Jesus Martin, R. Cetin-Atalay, and V. Atalay, "DEEPred: Automated protein function prediction with multi-task feed-forward deep neural networks," *Sci. Rep.*, vol. 9, no. 1, May 2019, Art. no. 7344. [Online]. Available: <https://doi.org/10.1038/s41598-019-43708-3>
- [14] Y. Li et al., "DEEPred: Sequence-based enzyme EC number prediction by deep learning," *Bioinformatics*, vol. 34, no. 5, pp. 760–769, Oct. 2017. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btx680>
- [15] J. Hou, B. Adhikari, and J. Cheng, "DeepSF: Deep convolutional neural network for mapping protein sequences to folds," *Bioinformatics*, vol. 34, no. 8, pp. 1295–1303, Dec. 2017. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btx780>
- [16] S. A. Memon, K. A. Khan, and H. Naveed, "HECNet: A hierarchical approach to enzyme function classification using a Siamese triplet network," *Bioinformatics*, vol. 36, no. 17, pp. 4583–4589, May 2020. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btaa536>
- [17] K. A. Khan, S. A. Memon, and H. Naveed, "A hierarchical deep learning based approach for multi-functional enzyme classification," *Protein Sci.*, vol. 30, no. 9, pp. 1935–1945, Sep. 2021.
- [18] R. Concu and M. N. D. S. Cordeiro, "Alignment-free method to predict enzyme classes and subclasses," *Int. J. Mol. Sci.*, vol. 20, no. 21, 2019, Art. no. 5389. [Online]. Available: <https://www.mdpi.com/1422-0067/20/21/5389>
- [19] Z. Zou, S. Tian, X. Gao, and Y. Li, "mlDEEPred: Multi-functional enzyme function prediction with hierarchical multi-label deep learning," *Front. Genet.*, vol. 9, 2019, Art. no. 714. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/fgene.2018.00714>
- [20] M. AlQuraishi, "End-to-end differentiable learning of protein structure," *Cell Syst.*, vol. 8, no. 4, pp. 292–301.e3, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405471219300766>
- [21] A. W. Senior et al., "Improved protein structure prediction using potentials from deep learning," *Nature*, vol. 577, no. 7792, pp. 706–710, Jan. 2020. [Online]. Available: <https://doi.org/10.1038/s41586-019-1923-7>
- [22] R. M. Rao et al., "MSA transformer," in *Proc. 38th Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 8844–8856. [Online]. Available: <https://proceedings.mlr.press/v139/rao21a.html>
- [23] Y. Du, J. Meier, J. Ma, R. Fergus, and A. Rives, "Energy-based models for atomic-resolution protein conformations," 2020, *arXiv: 2004.13167*. [Online]. Available: <https://arxiv.org/abs/2004.13167>
- [24] J. Yang, I. Anishchenko, H. Park, Z. Peng, S. Ovchinnikov, and D. Baker, "Improved protein structure prediction using predicted interresidue orientations," in *Proc. Nat. Acad. Sci.*, vol. 117, no. 3, pp. 1496–1503, 2020. [Online]. Available: <https://www.pnas.org/doi/abs/10.1073/pnas.1914677117>
- [25] S. Biswas, G. Khimulya, E. C. Alley, K. M. Esvelt, and G. M. Church, "Low-N protein engineering with data-efficient deep learning," *Nature Methods*, vol. 18, no. 4, pp. 389–396, Apr. 2021. [Online]. Available: <https://doi.org/10.1038/s41592-021-01100-y>
- [26] A. Madani et al., "Progen: Language modeling for protein generation," *bioRxiv*, 2020. [Online]. Available: <https://www.biorxiv.org/content/early/2020/03/13/2020.03.07.982272>
- [27] I. Anishchenko et al., "De novo protein design by deep network hallucination," *Nature*, vol. 600, no. 7889, pp. 547–552, Dec. 2021. [Online]. Available: <https://doi.org/10.1038/s41586-021-04184-w>
- [28] K. K. Yang, Z. Wu, and F. H. Arnold, "Machine-learning-guided directed evolution for protein engineering," *Nature Methods*, vol. 16, no. 8, pp. 687–694, Aug. 2019. [Online]. Available: <https://doi.org/10.1038/s41592-019-0496-6>
- [29] S. Mazurenko, Z. Prokop, and J. Damborsky, "Machine learning in enzyme engineering," *ACS Catalysis*, vol. 10, no. 2, pp. 1210–1223, Jan. 2020. [Online]. Available: <https://doi.org/10.1021/acscatal.9b04321>
- [30] J. Y. Ryu, H. U. Kim, and S. Y. Lee, "Deep learning enables high-quality and high-throughput prediction of enzyme commission numbers," in *Proc. Nat. Acad. Sci.*, vol. 116, no. 28, pp. 13996–14001, 2019. [Online]. Available: <https://www.pnas.org/doi/abs/10.1073/pnas.1821905116>
- [31] M. L. Bileschi et al., "Using deep learning to annotate the protein universe," *Nature Biotechnol.*, vol. 40, no. 6, pp. 932–937, Jun. 2022. [Online]. Available: <https://doi.org/10.1038/s41587-021-01179-w>
- [32] D. Dohan, A. Gane, M. L. Bileschi, D. Belanger, and L. Colwell, "Improving protein function annotation via unsupervised pre-training: Robustness, efficiency, and insights," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, New York, NY, USA, 2021, pp. 2782–2791. [Online]. Available: <https://doi.org/10.1145/3447548.3467163>

- [33] A. Elnaggar et al., "ProtTrans: Toward understanding the language of life through self-supervised learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 10, pp. 7112–7127, Oct. 2022.
- [34] N. Brandes, D. Ofer, Y. Peleg, N. Rappoport, and M. Linial, "ProteinBERT: A universal deep-learning model of protein sequence and function," *Bioinformatics*, vol. 38, no. 8, pp. 2102–2110, Feb. 2022. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btac020>
- [35] T. Sanderson, M. L. Bileschi, D. Belanger, and L. J. Colwell, "Proteinformer: Deep networks for protein functional inference," *bioRxiv*, 2021. [Online]. Available: <https://www.biorxiv.org/content/early/2021/10/06/2021.09.20.461077>
- [36] Enzyme commission number. [Online]. Available: <https://www.abbea.com/enzyme-commission-number>
- [37] T. U. Consortium, "UniProt: The universal protein knowledgebase in 2021," *Nucleic Acids Res.*, vol. 49, no. D1, pp. D480–D489, Nov. 2020. [Online]. Available: <https://doi.org/10.1093/nar/gkaa1100>
- [38] A. MacDougall et al., "UniRule: A unified rule resource for automatic annotation in the UniProt knowledgebase," *Bioinformatics*, vol. 36, no. 17, pp. 4643–4648, May 2020. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btaa485>
- [39] D. H. Haft, J. D. Selengut, and O. White, "The TIGRFAMs database of protein families," *Nucleic Acids Res.*, vol. 31, no. 1, pp. 371–373, Jan. 2003.
- [40] B. E. Suzek, Y. Wang, H. Huang, P. B. McGarvey, C. H. Wu, and UniProt Consortium, "UniRef clusters: A comprehensive and scalable alternative for improving sequence similarity searches," *Bioinformatics*, vol. 31, no. 6, pp. 926–932, Mar. 2015.
- [41] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, Curran Associates, Inc., 2017, pp. 6000–6010. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>
- [42] A. Kolesnikov et al., "An image is worth 16x16 words: Transformers for image recognition at scale," 2020. [Online]. Available: <https://arxiv.org/abs/2010.11929>
- [43] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite bert for self-supervised learning of language representations," in *Proc. Int. Conf. Learn. Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=H1eA7AEtvS>
- [44] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, Minneapolis, MN, USA, 2019, pp. 4171–4186. [Online]. Available: <https://doi.org/10.18653/v1/n19--1423>
- [45] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv: 1910.01108*. [Online]. Available: <http://arxiv.org/abs/1910.01108>
- [46] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, "ELECTRA: Pre-training text encoders as discriminators rather than generators," 2020, *arXiv: 2003.10555*.
- [47] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," 2019, *arXiv: 1909.05858*.
- [48] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [49] Z. Dai, Z. Yang, Y. Yang, J. G. Carbonell, Q. V. Le, and R. Salakhutdinov, "Transformer-XL: Attentive language models beyond a fixed-length context," 2019, *arXiv: 1901.02860*. [Online]. Available: <http://arxiv.org/abs/1901.02860>
- [50] M. Lewis et al., "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," 2019, *arXiv: 1910.13461*.
- [51] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.
- [52] T. Wolf et al., "Huggingface's transformers: State-of-the-art natural language processing," 2019, *arXiv: 1910.03771*.
- [53] K. Lu, A. Grover, P. Abbeel, and I. Mordatch, "Pretrained transformers as universal computation engines," 2021, *arXiv:2103.05247*.
- [54] M. Salem, A. Keshavarzi Arshadi, and J. S. Yuan, "AMPDeep: Hemolytic activity prediction of antimicrobial peptides using transfer learning," *BMC Bioinf.*, vol. 23, no. 1, Sep. 2022, Art. no. 389. [Online]. Available: <https://doi.org/10.1186/s12859-022-04952-z>
- [55] W. Li and A. Godzik, "Cd-hit: A fast program for clustering and comparing large sets of protein or nucleotide sequences," *Bioinformatics*, vol. 22, no. 13, pp. 1658–1659, May 2006. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btl158>
- [56] L. Fu, B. Niu, Z. Zhu, S. Wu, and W. Li, "CD-HIT: Accelerated for clustering the next-generation sequencing data," *Bioinformatics*, vol. 28, no. 23, pp. 3150–3152, Dec. 2012.



Azwad Tamir received the BSc degree from the Bangladesh University of Engineering and Technology with a major in electronics. He is currently working toward the PhD degree with the Department of Electrical and Computer Engineering, University of Central Florida. He is currently working as a graduate research assistant and is expected to graduate by the summer of 2024. During his time as a PhD student, he has been involved in numerous research projects involving the application of Machine Learning in various fields ranging from 3D-IC Physical Design to Bioinformatics. Some of his projects include: Using Generative Pretrained Graphical Neural Networks to design a Tier partitioner for 3D IC Physical Design, Skin Cancer Detection using Vision Transformers, CNN, and Ensemble learning from Images of Skin lesions and demonstrating the advantages of Vision Transformers over CNN models in cross Transfer Learning, and Localizing and Detecting House numbers using CNN models from raw street view images. He is currently working on using Sequence-based Transformers models for Protein Annotations. His research interests include machine learning, computer vision, transfer learning, bioinformatics, and natural language processing.



Milad Salem received the PhD degree in computer engineering with a focus on machine learning from the University of Central Florida. During his research, he experimented with pre-trained models and transfer learning which includes transferring and using these models in related or unrelated tasks. The experiments proved empirically successful in improving the performance of the models, even when transfer learning was performed between different domains (e.g. image domain and signal domain) or different data distributions (e.g. proteins and antimicrobial peptides).

Afterward, he used pre-trained models and the dictionary of patterns held with these models to assist in the implementation of small-molecule drug discovery pipelines in Malaria and Cancer. His research interests include transfer learning, knowledge discovery, drug discovery, and language modeling.



Jiann-Shiun Yuan received the MS and PhD degrees from the University of Florida, Gainesville, in 1984 and 1988, respectively. From 1988 to 1989 he was with Texas Instruments, Incorporated. Since 1990 he has been a Faculty member of the University of Central Florida (UCF), where he is currently a professor and the site director of the NSF I/UCRC MIST Center. He is the author of *Semiconductor Device Physics and Simulation* (Plenum, 1998) and *SiGe, GaAs and InP Heterojunction Bipolar Transistors* (Wiley, 1999) and has authored 360 papers in journals and conference proceedings. He supervised 32 PhD dissertations and 35 MS theses with UCF. Since 1990, he has been conducting many research projects funded by the National Science Foundation, Motorola, Renesas, Tokyo Electron, Lucent Technologies, National Semiconductor, and the State of Florida. He is a member of Eta Kappa Nu and Tau Beta Pi. He is an editor of the *IEEE Transactions on Device and Materials Reliability* and a distinguished lecturer for the *IEEE Electron Devices Society*. He was the recipient of the 1993 Outstanding Engineering Educator Award, IEEE Florida Council; the 1995, 2004, 2010, 2015, and 2020 Teaching Awards, UCF; the 2003 and 2018 Research Awards, UCF; and the 2016 Pegasus Professor Award, highest academic honor given to senior faculty at UCF. His current research interests include power semiconductor device reliability and deep learning for biomedical applications.